

Word Count Error Log

1. Incorrect File Path

- **What happened?**

When I ran the program, it threw a `FileNotFoundException` because it couldn't locate the file specified in the code. I had written `"source.txt"` as the file path without realizing it needed either an absolute path or a valid relative path based on the project's folder structure.

- **What caused it?**

I didn't verify the location of the file before running the program. I assumed `"source.txt"` would automatically work since the file was in the project, but Java couldn't locate it because the file path was incomplete.

- **How did I fix it?**

I replaced the file path with a relative path that accurately points to the file, ensuring it works within the project folder structure.

Error Code:

```
10 String fileName = "source.txt"; |
```

Fixed Code:

```
10 String fileName = "../Chapter11/src/Mastery/source.txt"; |
```

2. Missing Braces for try Block

- **What happened?**

When I tried to compile the program, it gave me a syntax error because I forgot to add curly braces `{}` around the `try` block. Without braces, Java doesn't know where the block ends, so the code won't compile.

- **What caused it?**

I was rushing while writing the code and thought braces weren't required for single-line statements. However, Java requires braces around a `try` block to clearly define its scope.

- **How did I fix it?**

I added the missing braces, enclosing all the statements within the `try` block.

Error Code:

```

18 try
19     BufferedReader br = new BufferedReader(new FileReader(fileName));
20

```

Fixed Code:

```

18 try {
19     BufferedReader br = new BufferedReader(new FileReader(fileName));
20

```

3. Logic Error in Word Counting

- **What happened?**

The program ran but didn't count any words. When I reviewed the output, `wordCount` was always zero, even though the file wasn't empty. This made the program calculate incorrect results for the total and average word length.

- **What caused it?**

I wrote the wrong condition (`if (word.isEmpty())`) inside the loop. This condition checks for empty words, so it skipped all valid words in the file. As a result no words were counted.

- **How did I fix it?**

I updated the condition to check for non-empty words (`if (!word.isEmpty())`), ensuring only valid words were processed and counted.

Error Code:

```

if (word.isEmpty()) {
    wordCount++;
    totalLength += word.length();
}

```

Fixed Code:

```

if (!word.isEmpty()) {
    wordCount++;
    totalLength += word.length();
}

```

4. Unclear Exception Handling

- **What happened?**

When I encountered an issue reading the file, the program printed "An error occurred," but this message didn't help me identify what went wrong. It didn't give any details about the type of error or how to fix it.

- **What caused it?**

I used a generic message in the `catch` block, which didn't include any specifics about the error. This made debugging harder because I couldn't see what caused the issue.

- **How did I fix it?**

I replaced the generic message with a detailed explanation of what might have gone wrong and included instructions for the user to check the file path and permissions.

Error Code:

```
37 } catch (IOException e) {  
38     System.out.println("An error occurred.");  
39 }
```

Fixed Code:

```
catch (IOException e) {  
    System.out.println("Error reading file. Please check the file path and permissions.");  
}
```